

AWS Mini Project

Ana-Maria Paraschivoiu

First step was to access AWS from the access portal.

AWS access portal

[Accounts](#) | [Applications](#)

AWS accounts (1)

Account Name	Account ID
▼ ITC BD Training	743737183475
AdministratorAccess	Access keys

Console home [Info](#)

Recently visited [Info](#)

S3	Athena
IAM	AWS Glue
Database Migration Service	Amazon Braket
CloudWatch	
EC2	
Lambda	
Kinesis	
Simple Notification Service	

General purpose buckets (48) [Info](#)

Copy ARN

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

I went to S3 in order to create two buckets, Source S3 bucket, which I named **ana-mini-project**, in this bucket the employee files will be uploaded manually, for the mini project. S3 jobs is to store the incoming employee data. No change was needed for the default settings, so they were left the same.

Second bucket created is **ana-mini-project-dataaggregate-s3**, which will be the output bucket. It is storing the result after Lambda has processed the employee data.

General configuration

AWS Region

Europe (London) eu-west-2

Bucket namespace

Choose the namespace where you want to create your bucket. [Learn more](#)

☒ Global namespace

By default, S3 creates general purpose buckets in the global namespace.

☐ Account Regional namespace (recommended)

General purpose buckets created in your account Regional namespace are ur

Bucket name [Info](#)

ana-mini-project

Bucket names must be 3 to 63 characters and unique within the global namespa

Copy settings from existing bucket - *optional*

Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs)

Object Ownership

☒ ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ ACLs enabled

Objects in this bucket can be owned by bucket and its objects can be specified

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, and public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block Public Access to prevent public access to buckets and objects. If you require some level of public access to this bucket or objects within, you can c

☒ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☒ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access resources using ACLs.

☒ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ Block public access to buckets and objects granted through *new* public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change existing public access.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

- ☒ Disable
☐ Enable

Tags - optional

You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

i You can use `s3:ListTagsForResource`, `s3:TagResource`, and `s3:UntagResource` APIs to manage tags. For a seamless transition, please provide permissions to `s3:ListTagsForResource`, `s3:TagResource`,

No tags associated with this bucket.

Add new tag

You can add up to 50 tags.

Key

project

Value - optional

mini-project1

Add new tag

You can add up to 49 more tags.

☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

- ☐ Disable
☒ Enable

▼ Advanced settings

Object Lock

Store objects using a write-once-read-many (WORM) model to help you prevent objects from being deleted or overwritten for a fixed amount of time or indefinitely. Object Lock works only in versioned buckets. [Learn more](#)

- ☒ Disable
☐ Enable

Permanently allows objects in this bucket to be locked. Additional Object Lock configuration is required in bucket details after bucket creation to protect objects in this bucket from being deleted or overwritten.

i Object Lock works only in versioned buckets. Enabling Object Lock automatically enables Versioning.

i After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket

ana-mini-project Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (0)

Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▾

Create folder

Upload

Find objects by prefix

< 1 >

⚙

Name

Type

Last modified

Size

Storage class

No objects

You don't have any objects in this bucket.

Upload

ana-mini-project-dataaggregate-s3 Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (0)

Copy S3 URI

Copy URL

Download

Open ↗

Delete

Actions ▾

Create folder

Upload

Find objects by prefix

< 1 >

⚙

Name

Type

Last modified

Size

Storage class

A new function was created, named **AnaLambdaMiniProject**. Because the Lambda code is written into Python, I chose at Runtime, Python 3.14.

Services



Lambda

Run code without thinking about servers

Actions ▾

Create function

Basic information

Function name

Enter a name that describes the purpose of your function.

AnaLambdaMiniProject

Function name must be 1 to 64 characters, must be unique to the Region, ar

Runtime Info

Choose the language to use to write your function. Note that the console co

Python 3.14

Permissions

By default, Lambda will create an execution role with permissions

In

Cancel

Create function

Lambda is using an IAM communication role to give it permission to access S3 and to write logs into CloudWatch. **AmazonS3FullAccess** gives Lambda permission to work with S3. So this permission is what it allowed Lambda to read the input file and write the output file. **AWSKambdaBasicExecutionRole**, is mainly for creating and writing logs and errors, so it sends logs to CloudWatch for monitoring and debugging.

Execution role

Role name

[AnaLambdaMiniProject-role-160vz783](#) ↗

☒	 AmazonS3FullAccess	AWS managed	Provides full access to all buckets via t...
☐	 AmazonS3ObjectLambdaExecutionRolePolicy	AWS managed	Provides AWS Lambda functions permi...
☐	 AmazonS3OutpostsFullAccess	AWS managed	Provides full access to Amazon S3 on ...
☐	 AmazonS3OutpostsReadOnlyAccess	AWS managed	Provides read only access to Amazon S...
☐	 AmazonS3ReadOnlyAccess	AWS managed	Provides read only access to all bucket...
☐	 AmazonS3TablesFullAccess	AWS managed	Provides full access to all S3 table buc...
☐	 AmazonS3TablesLakeFormationServiceRole	AWS managed	This managed policy grants AWS Lake ...
☐	 AmazonS3TablesReadOnlyAccess	AWS managed	Provides read only access to all S3 tabl...

Cancel

Add permissions

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

Q Search

☐ | Policy name ↗

☐

 [AmazonS3FullAccess](#)

☐

 [AWSLambdaBasicExecutionRole-5e2df231-3b58-4346-9...](#)

After creating the Lambda function, I opened the Cod tab, pasted the Python code. I made two small changes before deploying it: I changed the input file name to match employee1.csv and I changed the output location so the aggregate result is written to my separate target S3 bucket.

The screenshot displays the AWS Lambda console interface. At the top, there are tabs for Code, Test, Monitor, Configuration, Aliases, and Versions. The 'Code source' tab is active, showing a file explorer on the left with 'ANALAMBDAMINIPROJECT' and 'lambda_function.py'. The main area shows the Python code for 'lambda_function.py'.

```
1 import csv
2 import json
3 import boto3
4 import time
5 from datetime import datetime
6
7
8 def lambda_handler(event, context):
9
10     event_params = event["Records"][0]
11
12     bucket = event_params["s3"]["bucket"]["name"]
13     key = event_params["s3"]["object"]["key"]
14
15     print("here")
16
17     s3_resource = boto3.resource('s3')
18     s3_object = s3_resource.Object(bucket, key)
19
20     data = s3_object.get()['Body'].read().decode('utf-8').splitlines()
```

Below the code editor, the 'Function overview' section is visible, showing options for 'Diagram' and 'Template', and a button to '+ Add trigger'.

I added the S3 trigger, so that whenever a new employee file is uploaded to the source S3 bucket, the Lambda function starts automatically and processes the file. At the Event type, I chose **All object created event**, which means that whenever a new file/object is created, S3 automatically triggers the Lambda function.

Trigger configuration [Info](#)



S3

aws asynchronous storage

Bucket

Choose or enter the ARN of an S3 bucket that serves as the event source

🔍 s3/ana-mini-project

Bucket region: eu-west-2

Event types

Select the events that you want to have trigger the Lambda function. You can select multiple event types, but they must all match the same object key.

All object create events ✕

Cancel

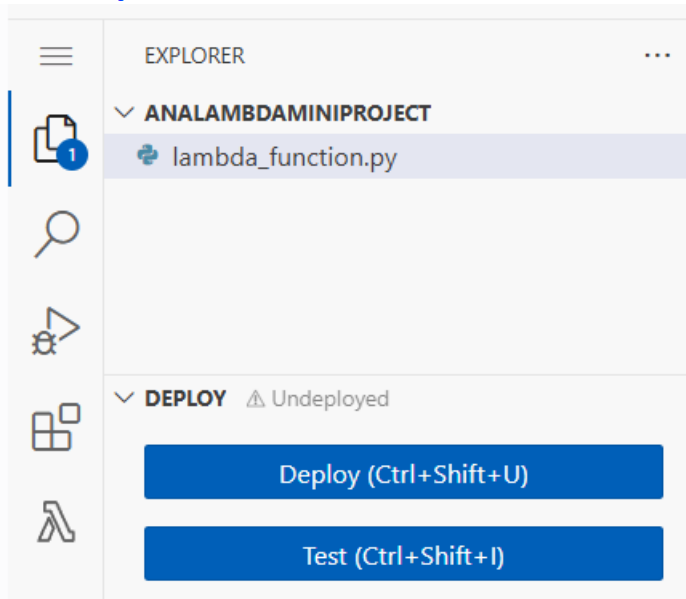
Add

```
if key == 'employee3.csv':  
    s3 = boto3.client('s3')  
    s3.put_object(Body=file_content, Bucket=bucket, Key='agg')
```

Amazon Q Tip 1/3: Start typing to

```
if key == 'employee1.csv':  
    s3 = boto3.client('s3')  
    s3.put_object(Body=file_content, Bucket=bucket, Key='agg')
```

Amazon Q Tip 1/3: Start typing to



🟢 Successfully updated the function "AnaLambdaMiniProject".

The file was uploaded into the Input bucket

Files and folders (1 total, 708.0 B)

 Find by name

Name	Folder
employee1.csv 	-

In order to monitor and debug the Lambda function, I used CloudWatch. Lambda execution logs and print statements were send to CloudWatch . I checked the recent log stream to identify errors when the pipeline wasn't completing successfully. So Lambda does the work and CloudWatch tells me what happened.

 **S3**

[+ Add trigger](#)

CodeTest**Monitor**Configuration

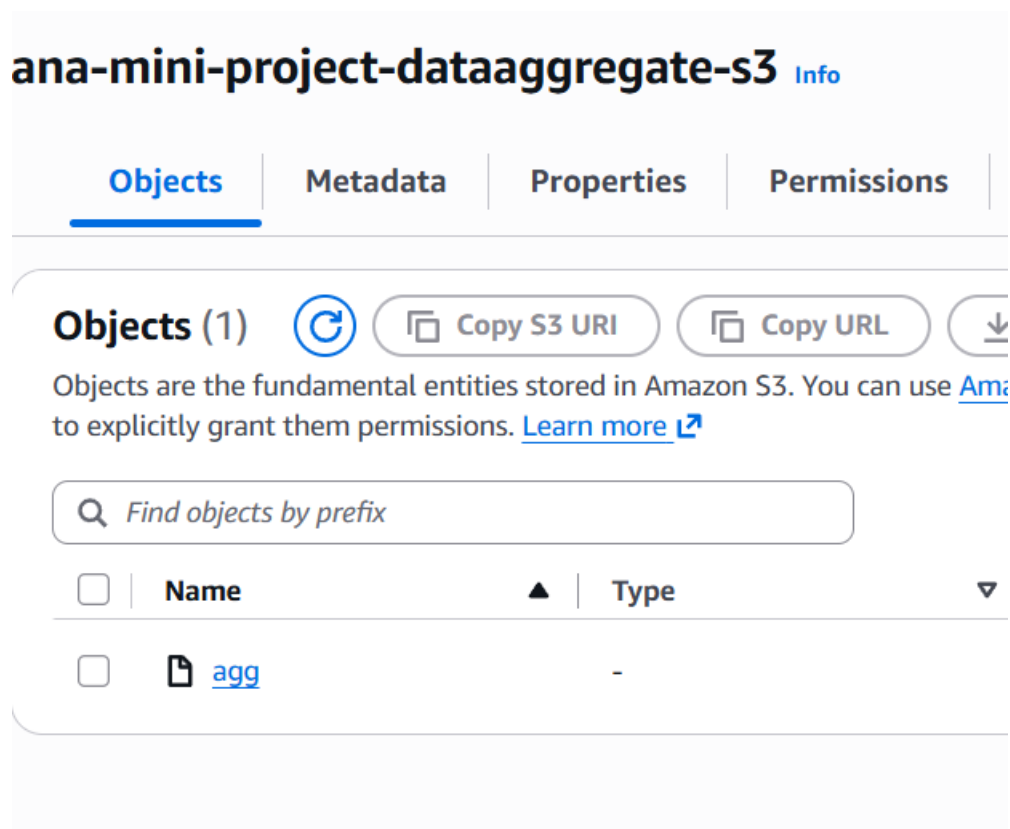
[View CloudWatch logs](#) 

Monitor [Info](#)

[Search all log streams](#)

▶	Timestamp	Message
▶	2026-08-09T19:35:57.847Z	REPORT RequestId: ef604c38-2ba1-43c3-8d36-3be6bf45083d Duratio...
▶	2026-08-09T19:35:59.172Z	START RequestId: 525732e5-88dd-4a12-bdff-b944b45430de Version:...
▶	2026-08-09T19:35:59.172Z	here
▶	2026-08-09T19:35:59.485Z	<_csv.reader object at 0x7fce02eaceb0>
▶	2026-08-09T19:35:59.485Z	headers: ['total india', 'us salary spend is ', '(260000', ' 4...
▶	2026-08-09T19:35:59.485Z	[]
▶	2026-08-09T19:35:59.485Z	total india salary spend is 0
▶	2026-08-09T19:35:59.485Z	total india salary spend is 0
▶	2026-08-09T19:35:59.485Z	total india,us salary spend is ,(0, 0)
▶	2026-08-09T19:35:59.526Z	END RequestId: 525732e5-88dd-4a12-bdff-b944b45430de
▶	2026-08-09T19:35:59.526Z	REPORT RequestId: 525732e5-88dd-4a12-bdff-b944b45430de Duratio...

When I uploaded the csv file into the source S3 bucket, the S3 event automatically triggers Lambda. Lambda reads the file, aggregates the salary by country, CloudWatch records the execution logs, and Lambda writes the aggregated result to the target S3 bucket.



Conclusions:

In this mini-project, I created a simple AWS data pipeline using **S3, Lambda, IAM and CloudWatch**. I uploaded an employee CSV file into an S3 bucket, which automatically triggered the Lambda function. The Lambda function read the employee data and calculated the total salaries based on the country. I used **IAM** to give Lambda permission to access S3, and **CloudWatch** to check the logs and find any errors. Finally, the aggregated salary result was saved into a separate S3 bucket. This project helped me understand how different AWS services can work together to create a simple automated data pipeline.